Lezione 4 - Protocollo HTTP 2 parte

Introduzione

Nella scorsa lezione abbiamo affrontato il concetto di protocollo, il modello TCP/IP e ISO/OSI sia separatamente che insieme e come i vari livelli di questi modelli, sia a livello pratico che concettuale, lavorano e comunicano tra loro.
Ora approfondiamo meglio il Protocollo HTTP.

HTTPS – HyperText Transfer Protocol Secure

- Definizione:
  - **non è un protocollo nuovo, ma una versione di HTTP con sicurezza aggiuntiva fornita da SSL/TLS.**
- **Funzionamento:**
  - **HTTP si occupa della trasmissione di pagine web.**
  - Con HTTPS, **tutte le comunicazioni tra client (browser) e server vengono crittografate, impedendo che possano essere intercettate o modificate da terzi**.
- **Come si riconosce:**
  - **Presenza del prefisso** `https://` **nell'URL.**
  - **Icona del lucchetto (🔒) nella barra degli indirizzi del browser.**

Livello di sicurezza

- **SSL (Secure Sockets Layer):**
  - **prima versione, oggi obsoleta e insicura.**
- **TLS (Transport Layer Security):**
  - **versione più recente e sicura, attuale standard.**

> 🖊 **Nota:**
>
> Sebbene oggi si usi quasi esclusivamente TLS, si continua a parlare di "**certificato SSL**" per consuetudine.

HTTPS – Le 3 garanzie di sicurezza fondamentali

HTTPS fornisce tre garanzie di sicurezza fondamentali:

1. **Crittografia (Encryption)** 🔒
   - **Rende i dati illeggibili a chi intercetta la comunicazione.**
   - Funziona grazie a **2** meccanismi combinati:
     - **Crittografia Asimmetrica (chiave pubblica/privata): usata all'avvio per scambiare in modo sicuro la chiave di sessione.**
     - **Crittografia Simmetrica (chiave condivisa): più veloce; una volta stabilita la chiave segreta, viene usata per crittografare tutto il traffico della sessione.**
2. **Autenticazione (Authentication)** ✅
   - **Garantisce che il client stia parlando con il server giusto (es. google.com) e non con un impostore.**
   - **Si basa sul certificato digitale SSL/TLS emesso da un'autorità di certificazione (CA).**
3. **Integrità (Integrity)** 🛡️
   - **Assicura che i dati non vengano alterati durante il transito.**
   - **Ogni messaggio include un Message Authentication Code (MAC), cioè una sorta di "firma digitale" che il destinatario può verificare.**

Il certificato SSL/TLS Handshake

Il processo che stabilisce una connessione sicura è chiamato **TLS Handshake**. Esattamente come per i due passaggi (1. TCP Connection Establishment (Three-Way Handshake), 2. TCP Connection Termination (Four-Way Handshake)) dell'handshake TCP/IP a livello di trasporto, anche qui due dispositivi (di solito un **client** e un **server**) devono eseguire una serie di passaggi per avviare una comunicazione sicura:

1. **Client Hello** 👋
   - **Il browser (client) contatta il server.**
   - **Comunica le versioni TLS e gli algoritmi di crittografia che supporta.**
2. **Server Hello** 📜
   - **Il server risponde scegliendo la versione TLS e l'algoritmo.**
   - **Invia il suo Certificato SSL/TLS e la chiave pubblica.**
3. **Verifica del Certificato** ✅
   - Il browser controlla che il certificato:

    - **non sia scaduto,**
    - **sia valido per il dominio richiesto,**
    - **sia firmato da una Certificate Authority (CA) fidata.**
4. **Scambio della Chiave di Sessione** 🔒
    - **Se tutto è ok, il browser genera una chiave segreta di sessione.**
    - **La crittografa con la chiave pubblica del server e la invia.**
    - Esistono diversi metodi di scambio chiavi (es. RSA, Diffie-Hellman, ECDHE), ma lo scopo è sempre lo stesso: stabilire una chiave condivisa in modo sicuro.
5. **Sessione Sicura** 🚀
    - **Solo il server, con la sua chiave privata, può decifrare.**
    - **Ora client e server condividono la stessa chiave segreta.**
    - **Da qui in poi tutta la comunicazione avviene con crittografia simmetrica (più veloce).**

📋 In sintesi:

TLS Handshake usa la **crittografia asimmetrica** solo per stabilire la chiave iniziale, poi passa alla **crittografia simmetrica** per scambiare i dati in modo sicuro ed efficiente.

📋 Ottenere un certificato TLS/SSL: Let's Encrypt

Per abilitare **HTTPS**, è indispensabile avere un **certificato TLS/SSL**.
Un tempo questo processo era **manuale e costoso**, ma oggi grazie a **Let's Encrypt** è diventato gratuito e automatizzato.

Cos'è Let's Encrypt?

- È una **Certificate Authority (CA)** globale e **non-profit**.
- Missione: rendere il web un luogo più sicuro per tutti.
- Supportata da grandi organizzazioni come **EFF, Mozilla, Cisco, Google Chrome**, ecc.

Caratteristiche principali:

- **Gratuito** → chiunque possieda un dominio può ottenere un certificato affidabile senza costi.

- **Automatizzato** → richiesta, convalida e rinnovo gestiti da software tramite il >protocollo **ACME** (Automated Certificate Management Environment).

Sicuro → stessi standard crittografici dei certificati a pagamento.

Durata Breve (90 giorni) → scelta di sicurezza:

- favorisce l'automazione del rinnovo,
- riduce i rischi in caso di compromissione della chiave.

Certificati tradizionali a pagamento

I certificati SSL/TLS **commerciali** sono rilasciati da **Certificate Authority (CA)** come:

- **DigiCert**
- **Sectigo (ex Comodo)**
- **GlobalSign**
  … e molte altre.

🔑 Caratteristiche principali:

- **A pagamento** → canone annuale.
- **Gamma più ampia di prodotti**:
  - certificati convalidati sul dominio (DV),
  - convalidati sull'organizzazione (OV),
  - convalidati estesi (EV, con la "barra verde" nei browser più datati).
- Offrono **servizi aggiuntivi**: garanzie economiche, supporto tecnico, soluzioni aziendali avanzate.

🌍 Certificati e Nomi di Dominio

- I certificati SSL/TLS **si legano ai nomi di dominio**, non agli indirizzi IP.
- Il browser verifica che il nome nel certificato corrisponda a quello del sito visitato.
- Se non c'è corrispondenza → ⚠️ errore di sicurezza.

📄 Perché non sugli IP?

- Con l'**hosting condiviso** un solo IP ospita centinaia di siti → legare i certificati agli IP sarebbe impraticabile.
- 🔧 La soluzione è **SNI (Server Name Indication)**:
  - **Estensione di TLS.**
  - **Permette al browser di comunicare al server il dominio richiesto.**
  - **Il server risponde presentando il certificato corretto.**

I livelli di validazione dei certificati: DV, OV ed EV

Quando una Certificate Authority (CA) emette un certificato, esegue un processo di verifica per assicurarsi che il richiedente sia chi dice di essere. Il livello di approfondimento di questa verifica determina il tipo di certificato.

1. **Domain Validation(DV):**

- **Cos'è**
  - **È il livello più semplice e diffuso di certificato SSL/TLS.**
  - **La CA verifica solo il controllo del dominio, non l'identità reale del proprietario.**
- **Come funziona la verifica**
  - Tutto il processo è **automatizzato**.
  - **La CA controlla che il richiedente abbia il dominio in gestione, ad esempio:**
    - Rispondendo a un'email inviata a indirizzi standard (es. `admin@dominio.it`
- **Caratteristiche**
  - **Veloce**: **ottenibile in pochi minuti.**
  - **Economico (spesso gratuito)**: es. Let's Encrypt.
  - **Garantisce crittografia sicura del traffico.**
  - ❌ Non fornisce alcuna informazione sull'identità dell'organizzazione (solo il dominio è verificato).

✓ Chi lo usa

- Perfetto per:
  - Blog personali,
  - Piccoli siti web,
  - Startup e piccole imprese,

- Qualsiasi sito dove **basta la cifratura**, senza necessità di garantire l'identità legale.

2. **Organization Validation (OV):**

- **Cos'è**
  - **Include tutte le verifiche del DV (controllo del dominio).**
  - **Aggiunge una verifica manuale dell'organizzazione da parte della CA.**
  - Un operatore umano controlla l'esistenza legale e la legittimità dell'organizzazione tramite:
    - Registri pubblici,
    - Database aziendali,
    - Documentazione ufficiale fornita dal richiedente.

📋 Cosa mostra il certificato

- Nome verificato dell'organizzazione,
- Località (città e paese).
- Queste informazioni sono visibili cliccando sul lucchetto nella barra del browser.

✓ Perché si usa

- Aumenta la **fiducia degli utenti**, perché dimostra che dietro al sito c'è una **organizzazione legale e registrata**, non un anonimo.

ⓘ Chi lo usa

- Aziende,
- Siti di e-commerce,
- Qualsiasi organizzazione che voglia un livello di **fiducia maggiore rispetto al DV**.

3. **Extended Validation (EV)**

- **Cos’è**
  - **Livello di validazione più rigoroso e standardizzato.**
  - La CA effettua un controllo approfondito dell’organizzazione, secondo linee guida severe, verificando:
    - Stato legale dell’azienda,
    - Stato operativo e fisico,
    - Autorizzazione interna alla richiesta del certificato.

📋 Cosa mostra il certificato

- In passato, i browser evidenziavano i siti con EV tramite la **barra verde** nella barra degli indirizzi con il nome dell’azienda.
- Oggi questa evidenziazione visiva è quasi scomparsa, ma le **informazioni dettagliate sull’identità legale** rimangono accessibili cliccando sul lucchetto.

✓ Perché si usa

- Offre il **massimo livello di fiducia** sull’identità del sito.
- Protegge efficacemente gli utenti dal **phishing**, perché un malintenzionato non potrebbe superare questa verifica così rigorosa.

ⓘ Chi lo usa

- Banche, istituzioni finanziarie,
- Grandi piattaforme di e-commerce,
- Agenzie governative,
- Qualsiasi sito dove la **fiducia dell’utente** è fondamentale.

Confronto Let's Encrypt vs Certificati Tradizionali

1. **Costo**
   - Let's Encrypt: totalmente gratuito.
   - Tradizionali: a pagamento, prezzo variabile in base al tipo di certificato e al fornitore.
2. **Processo di ottenimento e automazione**

- Let's Encrypt: completamente automatico tramite protocollo ACME; client software gestisce richiesta, convalida e rinnovo.
- Tradizionali: spesso manuale o semi-manuale; richiede generazione del CSR, invio alla CA e installazione manuale.

3. **Periodo di validità**
   - Let's Encrypt: 90 giorni; breve durata che incentiva l'automazione e aumenta la sicurezza.
   - Tradizionali: tipicamente 1 anno; riduce la frequenza dei rinnovi manuali.
4. **Livelli di validazione**
   - Let's Encrypt: solo Domain Validation (DV), verifica automatica del controllo del dominio.
   - Tradizionali: DV, Organization Validation (OV) e Extended Validation (EV); includono anche verifica dell'identità legale dell'organizzazione.
5. **Garanzia e supporto tecnico**
   - Let's Encrypt: nessuna garanzia finanziaria; supporto tramite comunità e documentazione.
   - Tradizionali: offrono garanzie monetarie in caso di emissione errata e supporto tecnico dedicato (telefono, chat, email).

Quale scegliere tra i due?

Quando scegliere Let's Encrypt

- Siti personali, blog, portfolio o siti di piccole imprese.
- Protezione di API, servizi backend o ambienti di sviluppo/staging.
- Preferenza per l'automazione, evitando la gestione manuale dei certificati.
- Budget limitato o priorità alla gratuità.

Quando considerare un Certificato Tradizionale (OV/EV)

- Grandi siti di e-commerce, piattaforme di pagamenti, siti bancari o governativi.
- Necessità di garantire la fiducia dell'utente tramite verifica dell'identità legale.
- Bisogno di una garanzia finanziaria per protezione propria e dei clienti.
- Necessità di supporto tecnico dedicato.

Perché HTTPS è Fondamentale per le API REST?

Nel contesto delle API REST, usare HTTP semplice è un errore di sicurezza gravissimo. L'uso di HTTPS non è opzionale, è un requisito indispensabile

- **Protezione dei dati sensibili**
  Tutte le informazioni trasmesse (dati personali, credenziali, informazioni finanziarie o segreti aziendali) viaggiano nel corpo della richiesta o nei parametri. Senza HTTPS, sarebbero intercettabili in chiaro.
- **Sicurezza dei token di autenticazione**
  Token come Bearer Token o JWT vengono inviati negli header Authorization. Se intercettati su HTTP, un attaccante può impersonare l'utente e accedere alle risorse senza restrizioni.
- **Prevenzione di attacchi Man-in-the-Middle (MITM)**
  Su HTTP un malintenzionato potrebbe intercettare o modificare le richieste e le risposte (ad esempio alterando transazioni o iniettando dati malevoli). HTTPS protegge da questi attacchi.
- **Affidabilità e professionalità**
  Un'API esposta su HTTP denota trascuratezza nella sicurezza. Molti client moderni (browser, app mobile) bloccano le richieste non sicure, rendendo l'API inefficace o inutilizzabile.

Protocollo HTTP: descrizione

1. **Ruolo del server**
   - **Il server è il componente che offre servizi o risorse (ad esempio, dati su utenti, prodotti, post di un blog).**
   - **Rimane "in attesa" delle richieste dei client, pronto a rispondere ogni volta che ne arriva una.**
2. **Ruolo del client**
   - **Il client è colui che richiede un servizio al server. Può essere un'applicazione web, un'app mobile o un altro server.**
   - **Per richiedere un servizio, il client deve specificare dove e cosa vuole fare.**
3. **URL: dove andare**
   - **L'URL indica esattamente la risorsa sul server a cui il client vuole accedere.**

- Si compone di:
  - **protocollo** (es. `http` o `https`) → **indica come comunicare.**
  - **IP o dominio** → **identifica il server.**
  - **porta** → **opzionale, indica la porta del servizio sul server.**
  - **path/resource** → **indica la risorsa specifica, ad esempio `/users/123` per l'utente con ID 123.**
- Esempio completo:

```
http://192.168.1.10:8080/users/123
```

4. . **VERB (metodi HTTP): cosa fare**
   - **Il protocollo HTTP definisce alcune parole chiave (chiamate verbi o metodi) per indicare quale azione si vuole compiere sulla risorsa.**
   - In REST, i principali sono:
     - **GET** → **leggere la risorsa (es. ottenere dati di un utente)**.
     - **POST** → **creare una nuova risorsa (es. aggiungere un nuovo utente).**
     - **PUT** → aggiornare **tutta** la risorsa (es. modificare tutti i dati di un utente).
     - **PATCH** → **aggiornare parzialmente la risorsa (es. cambiare solo l'email di un utente).**
     - **DELETE** → **eliminare la risorsa (es. cancellare un utente).**

In pratica, il client dice al server:

> "Voglio fare questa azione (**VERB**) su questa risorsa (**URL**)".

Il server riceve la richiesta, la elabora e restituisce una risposta appropriata, spesso con dati in formato JSON o XML.

Struttura delle Request e delle Response HTTP

Ogni comunicazione HTTP — **sia in ingresso (request) sia in uscita (response)** — è composta da due parti fondamentali:

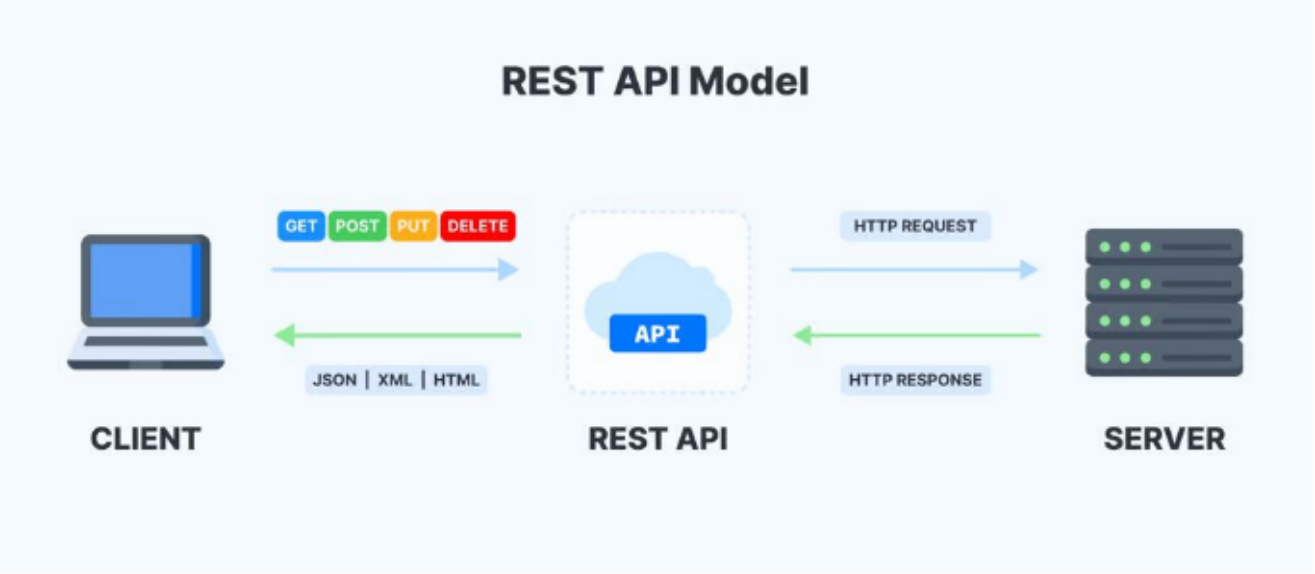
1. **Header**
   - Contiene i *metadati*:
     - **cioè informazioni strutturate che descrivono la richiesta o la risposta.**

Alcuni esempi: tipo di contenuto, metodi di autenticazione, lunghezza del body, lingua, cache, user-agent del client.

2. **Body**
   - **Contiene il *contenuto vero e proprio* trasportato.**
   - In passato si utilizzavano formati come **XML**.
   - Nelle architetture REST moderne, lo standard di fatto è **JSON**, grazie alla sua leggerezza e leggibilità.

Come il client può inviare dati al server

Il client dispone di **tre modalità principali** per trasmettere dati all'interno di una richiesta HTTP.
La scelta dipende dal tipo di azione e dal contesto applicativo.



Rest API Model.png

1. **Query String (parametri nella URL)**
   I dati vengono aggiunti dopo il simbolo `?` nella URL sotto forma di coppie chiave-valore.
   Esempio:

```shell
http://ip:port/path/resource?var1=aaa&var2=bbb
```

- **Viene tipicamente usata per filtrare, cercare o ordinare informazioni (operazioni idempotenti).**

2. **Parametri nel percorso della URL (Path Parameters)**
   I dati sono incorporati direttamente nella struttura del percorso.
   Esempio:

```shell
http://ip:port/path/resource/aaa/bbb
```

- **Usata generalmente per identificare in modo univoco una risorsa, come `/users/42/orders/3`.**

3. **Body della richiesta**
   - **I dati vengono inseriti nel corpo della request**.
     È il metodo preferito quando si inviano payload complessi (creazione o modifica di risorse), solitamente in formato JSON.
     Esempio:

```json
{
  "nome": "Luca",
  "email": "test@example.com"
}
```

Come il server invia dati al client

Il server ha un'unica modalità principale per restituire i dati:

- **inserirli nel body della response.**

Il body della risposta contiene l'informazione richiesta dal client, tipicamente anch'essa in formato **JSON** nelle API REST.

L'header della response, come nella request, trasporta invece i metadati (es. `Content-Type`, codice di stato HTTP, eventuali cookie, informazioni sulla cache, ecc.).

Status Code HTTP

Ogni volta che il server riceve una richiesta, è obbligato a restituire uno **status code:**

- **cioè un codice numerico che comunica al client l'esito dell'operazione.** Questi codici sono suddivisi **per fasce di 100**, e ogni fascia rappresenta una categoria di risposta.

1. Codici 2xx - Successo

Indicano che la richiesta è stata compresa, accettata ed eseguita correttamente.

- **200 – OK**
  - **Risposta standard per un'operazione conclusa con successo.**
  - **Viene usato soprattutto nella lettura di risorse (GET).**
- **201 – Created**
  - **Indica che una nuova risorsa è stata creata con successo.**
  - **È tipico delle operazioni POST.**
- **203 – Non-Authoritative Information**
  - **La richiesta è stata completata, ma le informazioni provengono da una copia non autorevole (proxy o cache).**
- **204 – No Content**
  - **L'operazione è avvenuta con successo, ma il server non invia alcun contenuto nel body della risposta**.
  - **Utile per operazioni di DELETE o update.**

2. Codici 4xx - Errori lato client

Segnalano che la richiesta contiene un errore o non rispetta le regole del server.
Sono problemi imputabili al client.

- **400 – Bad Request**
  - **La richiesta non è valida o è formata in modo errato (JSON malformato, parametri mancanti, sintassi sbagliata).**
- **401 – Unauthorized**
  L'autenticazione è richiesta, ma non è stata fornita o è invalida.
- **403 – Forbidden**
  - **Il server ha compreso la richiesta, ma rifiuta di autorizzarla.**
  - **Qui l'autenticazione può essere presente, ma i permessi non sono sufficienti.**
- **404 – Not Found**

    - **La risorsa richiesta non esiste o l'endpoint è errato.**
- **409 – Conflict**
    - **La richiesta non può essere completata perché in conflitto con lo stato attuale del server (ad esempio un valore che deve essere univoco).**

3. Codici 5xx - Errori lato server

Indicano che la richiesta era valida, ma il server non è riuscito ad elaborarla per un problema interno.

- **500 – Internal Server Error**
    - Errore generico:
        - **qualcosa è andato storto all'interno del server.**
- **501 – Not Implemented**
    - **Il server non riconosce o non supporta il metodo richiesto.**
- **502 – Bad Gateway**
    - **Il server sta operando come gateway o proxy, ma ha ricevuto una risposta non valida da un altro server a monte.**
- **503 – Service Unavailable**
    - **Il server è temporaneamente non disponibile (sovraccarico, manutenzione, ecc.).**
- **504 – Gateway Timeout**
    - **Il server, funzionando da proxy, non ha ricevuto una risposta in tempo dal server a monte.**
- **599 – Network Timeout**
    - **Errore non standard, usato da alcuni server e proxy per indicare un timeout di rete.**